

数据库(6)

孙甲松

sunjiasong@tsinghua.edu.cn

电子工程系 信息认知与智能系统研究所

罗姆楼6-104

电话: 13901216180/62796193

2025.10.

第4章 关系系统和查询优化

参考书 第5版 p.275 第9章 关系查询处理和查询优化

参考书 第6版 p.301 第10章 关系查询处理和查询优化

4.1 关系系统

- 4.1.1 关系系统的定义

- 4.1.2 关系系统的分类

- 4.1.3 全关系系统的十二条基本准则

4.2 关系数据库系统的查询优化

- 4.2.1 关系系统及其查询优化

- 4.2.2 一个例子

- 4.2.3 查询优化的一般准则

- 4.2.4 关系代数等价变换规则

- 4.2.5 关系代数表达式的优化算法

4.3 物理优化

- 4.3.1 基于启发式规则的优化

- 4.3.2 基于代价估算的优化

4.4 查询计划的执行

4.1 关系系统

- 关系模型的三个组成部分（模型三要素）：
 - 数据结构 域及域上定义的关系
 - 关系操作 关系代数的各种操作
 - 数据完整性 三种完整性(实体,参照,用户定义)
- 支持关系模型的系统，即为关系系统

4.1.1 关系系统的定义

一个系统可以定义为关系系统，当且仅当它支持：

1. 关系数据库

2. 选择、投影和(自然)连接运算

而且对这些运算不必要求定义任何物理存取路径（即数据的物理独立性）。

两条三运算缺一不可！

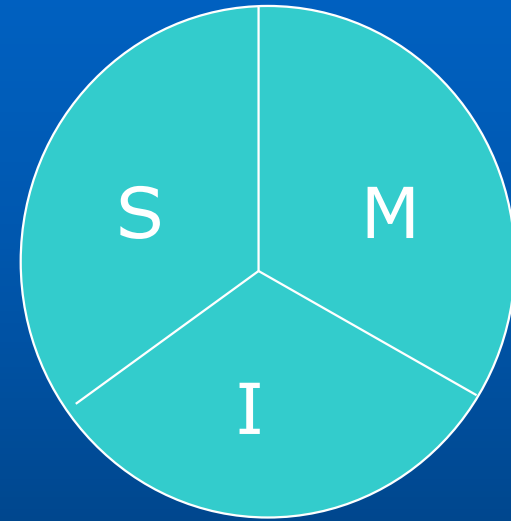
4.1.2 关系系统的分类

E.F.Codd分类:

S: 结构(Structure)

M: 数据操纵(Manipulation)

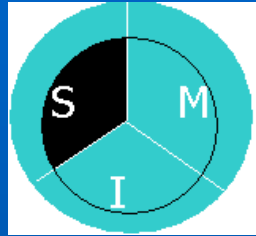
I: 完整性(Integrity)



4.1.2 关系系统的分类

分为4类:

1.表式系统



仅支持关系(表)数据结构, 不支持集合运算 不能算关系系统

2.(最小)关系系统



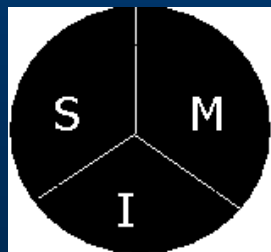
支持关系(表)数据结构, 和三种关系运算

3.关系完备的系统



支持关系(表)数据结构, 和全部关系运算

4.全关系的系统



支持关系模型的所有特征, 关系上完备, 支持域的概念, 支持实体完备性和参照完备性

4.1.2 关系系统的分类

举例：

1. 表式系统

2. (最小)关系系统

Foxbase, FoxPro, Access

3. 关系完备的系统

SQL_Server 2000, Sybase, Oracle 8

4. 全关系的系统

Oracle 11g, Oracle 12c、DB2高版本等已经接近全关系的系统。

4.1.3 全关系系统的十二条基本准则

E.F.Codd给出了全关系系统应遵守的十二条基本准则：

0. 一个关系型的DBMS必须能完全通过它的能力来管理数据库。（12条准则的基础）
1. 信息准则。关系型DBMS的所有信息都应在逻辑一级上用一种方法即表中的值显式地表示。
2. 保证访问准则。依靠表名、主码和列名的组合，保证能以逻辑方式访问关系数据库中的每个数据项。
3. 空值的系统化处理。全关系型的DBMS应该支持空值的概念，并用系统化的方式处理空值。
4. 基于关系模型的动态的联机数据字典。数据库的描述在逻辑级上应该和普通数据采用同样的表示方式，使得授权用户可以使用查询一般数据所用所谓关系语言来查询数据库的描述信息。

4.1.3 全关系系统的十二条基本准则

- 5.统一的数据子语言准则。一个关系系统可以具有几种语言和多种终端使用方式(QBE,cmd),但必须有一种语言能实现全部功能(DDL, DML, DCL等)。
- 6.视图更新准则。所有理论上可更新的视图也应该允许由系统更新。
- 7.高级的插入、修改和删除操作。
- 8.数据物理独立性。
- 9.数据逻辑独立性。
- 10.数据完整性的独立性。完整性约束条件必须用DB语言定义并存储在DD中。
- 11.分布独立性。
- 12.无破坏准则。若一个关系系统具有一个低级语言,则这个低级语言不能违背或绕过完整性准则。

4.2 关系数据库系统的查询优化

- 关系的查询优化是关系DBMS成败关键。
- 由于关系查询的优化，才使得现在的关系DBMS商用产品成为可能。
- 关系表达式语义级别很高，只说明“怎么干”，关系系统可以分析查询语义，并进行查询优化，以获得更高的效率。
- 目标是：选择有效的策略，求得给定关系表达式的值。

4.2.1 关系系统及其查询优化

- 实际的各个系统对查询优化的具体实现不尽相同，但一般可归纳为4个步骤：
 1. 将查询转换成某种内部表示，通常是语法树；
 2. 根据一定的等价变换规则把语法树转换成标准（优化）形式；
 3. 选择低层的操作算法。对语法树中的每一个操作根据存取信息来选择具体的执行算法；
 4. 生成物理查询计划，即查询执行方案。从多个查询执行方案中选择代价最小的一个：

总代价 = I/O代价 + CPU代价（+ 内存代价）

数据库服务器通常内存很大，以减小或忽略内存代价！

4.2.2 一个例子

例如：求选修了C2号课的学生姓名。

SELECT Sname

FROM S, SC

WHERE S.SNO=SC.SNO and SC.CNO='C2'

等价的关系代数式有多种：

$$\prod_{Sname} (\sigma_{S.SNO=SC.SNO \wedge SC.CNO='C2'} (S \times SC))$$

$$\prod_{Sname} (S \bowtie \sigma_{SC.CNO='C2'} (SC))$$

$$\prod_{Sname} (\sigma_{SC.CNO='C2'} (S \bowtie SC))$$

优化是必要的，通常先选择再连接。

4.2.3 查询优化的一般准则

1. 选择运算尽可能先做（以减少元组数）。
2. 在执行连接前对关系适当地预处理。

预处理方法有两种：

索引连接方法：在连接属性上先建立索引并进行排序

排序合并连接方法：先按连接属性进行排序，连接

只需要扫描一遍表即可

3. 把投影运算和选择运算同时进行。
4. 把投影同其前或其后的双目运算符结合起来，没有必要为了去掉某些字段而扫描一遍关系。
5. 把某些选择运算同在其前面要执行的笛卡尔积结合起来成为一个连接运算。连接运算效率高于笛卡尔积。
6. 找出公共表达式（以减少重复计算）。

4.2.4 关系代数等价变换规则

1. 连接与笛卡尔积的交换率
2. 连接与笛卡尔积的结合率
3. 投影的串接定律
4. 选择的串接定律
5. 选择和投影的交换率
6. 选择与笛卡尔积的交换率
7. 选择与并运算的交换
8. 选择与差运算的交换
9. 投影与笛卡尔积的交换
10. 投影与并运算的交换

4.2.5 关系代数表达式的优化算法

选择和投影尽可能早做！！！！以优化算法：

1. 利用规则4将 $\sigma_{F_1 \wedge F_2 \wedge \dots \wedge F_n}(E)$ 变换为 $\sigma_{F_1}(\sigma_{F_2}(\dots \sigma_{F_n}(E)\dots))$ 。
2. 对每一个选择操作，利用规则4-8尽可能移到语法树的树叶上（先做）。
3. 对每一个投影操作，利用规则3, 9, 10, 5尽可能移到语法树的树叶上（先做）。
4. 利用规则3—5把选择和投影的串接合并成单个操作。
5. 把上述得到的语法树的内节点分组。
6. 分结点组顺序计算。

4.3 物理优化

- 代数优化改变查询语句中操作的次序和组合，但不涉及底层的存取路径。对每一种操作有多种执行这个操作的算法，有多条存取路径，因此对于一个查询语句有多种存取方案，它们的执行效率不同，有的会相差很大。因此，仅仅进行代数优化是不够的。
- 物理优化就是要选择高效合理的操作算法或存取路径，求得优化的查询计划，最终达到查询优化的目标。
- 选择的方法可以是：
 - ① 基于启发式规则的优化。启发式规则是指那些在大多数情况下都适用，但不是在每种情况下都是最好的规则。
 - ② 基于代价估算的优化。使用优化器估算不同执行策略的代价，并选出具有最小代价的执行计划。
 - ③ 两者结合的优化方法。查询优化器通常会把这两种技术结合在一起使用。通常先使用启发式规则选取若干较优的候选方案，然后分别计算这些候选方案的执行代价，较快地选出最终的优化方案。

4.3.1 基于启发式规则的优化

下面以选择操作和连接操作为例，介绍常用的启发式规则。

1. 选择操作的启发式规则

对于小关系，使用全表顺序扫描，即使选择列上有索引。

对于大关系，启发式规则有：

- ①对于选择条件是“主码=值”的查询，查询结果最多是一个元组，可以选择主码索引扫描。一般的关系数据库管理系统会自动建立主码索引。
- ②对于选择条件是“非主属性=值”的查询，并且选择列上有索引，则要估算查询结果的元组数目，如果比例较小($<10\%$)可以使用索引扫描方法，否则还是使用全表顺序扫描。
- ③对于选择条件是属性上的非等值查询或者范围查询，并且选择列上有索引，同样要估算查询结果的元组数目，如果选择率 $<10\%$ 可以使用索引扫描方法，否则还是使用全表顺序扫描。

4.3.1 基于启发式规则的优化

- ④对于用**AND**连接的合取选择条件，如果有涉及这些属性的组合索引，则优先采用组合索引扫描方法；如果某些属性上有单属性索引，则可以用索引扫描方法；否则使用全表顺序扫描。
- ⑤对于用 **OR** 连接的析取选择条件，一般使用全表顺序扫描。

2. 连接操作的启发式规则

- ①如果两个表都已经按照连接属性排序，则选用排序-合并算法。
- ②如果一个表在连接属性上有索引，则可以选择索引连接算法。
- ③如果上面两个规则都不适用，其中一个表较小，则可以选择哈希连接算法。

4.3.1 基于启发式规则的优化

④最后可以选用嵌套循环连接算法，并选择其中较小的表，确切地讲是选择占用的块数(B)较少的表，作为外表(外循环的表)。其理由如下：

设连接表 R 与 S 占用的块数分别为 B_R 与 B_S ，连接操作使用的内存缓冲区块数为 K ，分配 $K-1$ 块给外表。如果 R 为外表，则嵌套循环连接算法存取的块数为

$B_R + B_R B_S / (K-1)$ ，显然应该选择块数小的表作为外表。

上面列出了一些主要的启发式规则，在实际的关系数据库管理系统中，启发式规则比上面这些要多得多。

4.3.2 基于代价估算的优化

- ◆ 启发式规则优化是定性的选择，比较粗糙，但是它实现简单，而且优化本身的代价较小，适合用于解释执行的系统。因为解释执行的系统，其优化开销包含在查询总开销之中。
- ◆ 在编译执行的系统中，一次编译优化，多次执行，查询优化和查询执行是分开的。因此，可以采用更精细复杂一些的基于代价估算的优化方法。

1、统计信息

基于代价估算的优化方法要计算各种操作算法的执行代价，它与数据库的状态密切相关。为此在数据字典中存储了优化器需要的数据库统计信息(**database statistics information**)，主要包括如下几个方面：

4.3.2 基于代价估算的优化

- ①对每个基本表，包括该表的元组总数(N)、元组长度(L)、占用的块数(B)等。
- ② 对基本表的每个列，包括该列不同值的个数(m)、该列最大值与最小值，该列是否已建立索引以及是哪种索引等。

根据这些统计信息，可以计算出谓词条件的选择率(f)，如果不同值的分布是均匀的：

$$f = 1/m;$$

如果不同值的分布不均匀，则要计算每个值的选择率：

$$f = \text{具有该值的元组数}/N$$

- ③ 对索引，例如 **B+**树索引，包括该索引的层数(L)、不同索引值的个数、索引的选择期数 S (有 S 个元组具有某个索引值)、索引的叶结点数(Y)等。

4.3.2 基于代价估算的优化

2、代价估算示例

下面给出若干操作算法的执行代价估算。所用符号 N 、 B 、 m 、 f 、 L 、 Y 等的含义如前所述，其中下标表示关系名，例如， N_R 表示关系 R 的元组数， B_R 表示关系 R 占用的块数，以此类推。

(1)全表扫描算法的代价估算

①如果基本表大小为 B 块，全表扫描算法的代价 $\text{cost}=B$ ，满足条件的元组占用 $f*B$ 块。

②如果选择条件是“码=值”，那么平均搜索代价 $\text{cost}=B/2$ ，且最多只有一个元组满足条件。

(2)索引扫描算法的代价估算

①如果选择条件是“码=值”，则采用该表的主码索引。若为 $B+$ 树，层数为 L ，需要存取 $B+$ 树中从根结点到叶结点 L 块，再加上基本表中该元组所在的那一块，所以 $\text{cost}=L+1$ 。

4.3.2 基于代价估算的优化

②如果选择条件涉及非码属性，若为B+树索引，选择条件是=比较操作，索引的选择基数为 N/m (有 N/m 个元组满足条件)。因为满足条件的元组可能会保存在不同的块上，所以(最坏的情况): $\text{cost} = L + N/m$ 。

③ 如果比较条件是 $>$ ， $>=$ ， $<$ ， $<=$ 操作，假设有一半的元组满足条件，那么就要存取一半的叶子结点，并通过索引访问一半的表存储块。所以， $\text{cost} = L + Y/2 + B/2$ 。如果可以获得更准确的选择基数，则可以进一步修正 $Y/2$ 与 $B/2$ 。

(3)嵌套循环连接算法的代价估算

前面提到过，嵌套循环连接算法的代价为：

$$\text{cost} = B_R + B_R B_S / (K-1)$$

如果需要把连接结果写回磁盘，则：

$$\text{cost} = B_R + B_R B_S / (K-1) + (F_{RS} * N_R * N_S) / M_{RS}$$

其中： F_{RS} 为连接选择率，表示连接结果元组数的比例；

M_{RS} 是存放连接结果的块因子，表示每块中可以存放的结果元组数目。

4.3.2 基于代价估算的优化

(4) 排序-合并连接算法的代价估算

①如果连接表已经按照连接属性排好序，则：

$$\text{cost} = B_R + B_S + (F_{RS} * N_R * N_S) / M_{RS}$$

②如果必须对文件排序，那么还需要在代价函数中加上排序的代价。对于包含**B**个块的文件，其外排序的代价大约是**4B**，包括按照可用内存大小分段读入文件(代价为**B**)，排成有序子表并写出去(代价为**B**)，最后将所有的有序子表归并成全局有序文件并写回磁盘(代价为**2B**)。因此总代价为：

$$\text{cost} = 5B_R + 5B_S + (F_{RS} * N_R * N_S) / M_{RS}$$

③在内存足够大的情形下，也可以不生成全局有序文件，而是直接在有序子表上进行合并，这样可以节省**2B**的代价。

上面仅仅列出了少数操作算法的代价估算示例。在实际的关系数据库管理系统中代价估算公式要比上面这些多很多，也复杂得多。

4.4 查询计划的执行

- 查询优化完成后，关系数据库管理系统为用户查询生成了一个查询计划。该查询计划的执行可以分为**自顶向下**和**自底向上**两种方式。
- 在**自顶向下**的执行方式中，系统反复向查询计划顶端的操作符发出需要查询结果元组的请求，操作符收到请求后就试图计算下一个(几个)元组并返回这些元组。在计算时，如果操作符的输入缓冲区为空，它就会向其子操作符发送需求元组的请求.....这种需求元组的请求会一直传到叶子结点，启动叶子操作符运行，并返回其父操作符一个(几个)元组，父操作符再计算自己的输出返回给上层操作符，直至顶端操作符。重复这一过程，直到处理完整个关系。

4.4 查询计划的执行

- 在**自底向上**的执行方式中，查询计划从叶子结点开始执行，叶子结点操作符不断产生元组并将它们放入其输出缓冲区，直到缓冲区填满为止，这时它必须等待其父操作符将元组从该缓冲区中取走才能继续执行。然后其父结点操作符开始执行，利用下层的输入元组来产生它自己的输出元组，直到其输出缓冲区满为止。这个过程不断重复，直到产生所有的输出元组。
- 很明显，自顶向下的执行方式是一种被动的、需求驱动的执行方式，而自底向上的执行方式是一种主动的执行方式。
- 查询处理是关系数据库系统的核心，而查询优化技术又是查询处理的关键技术。因此需要我们知晓。